INSTITUTO FEDERAL
Catarinense
Campus Camboriú

Desenvolvimento Web II

Professor: Euclides Paim

euclides.paim@ifc.edu.br

INSTITUTO FEDERAL
Catarinense
Campus Camboriú

Controle de Versão
Git

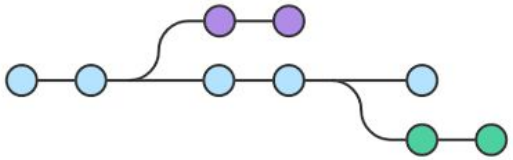
Professor: Euclides Paim

euclides.paim@ifc.edu.br

INSTITUTO FEDERAL
Catarinense
Campus Camboriú

Sumário

- Introdução
  - Sistemas Locais de Controle de Versão
  - Sistemas Centralizados de Controle de Versão
  - Sistemas Distribuídos de Controle de Versão
- Git
  - Uma Breve História do Git
  - O básico do Git
  - Os três estados
  - As três seções principais
  - O fluxo de trabalho
  - Linha de comando
  - Instalação
  - Configuração inicial
  - Fundamentos
- *Branching*
  - Criando um novo Branch
  - Alternando entre Braches
  - Mesclando (merge) Branches
- GitHub
  - Configurando uma conta
  - Fazendo *Fork* de projetos
  - O fluxo do Github



Controle de Versão

Introdução

O que é "**Controle de Versão**"?

- É um <u>sistema</u> que registra alterações em um arquivo ou conjunto de arquivos ao longo do tempo.
- Possibilita acesso a versões específicas deste arquivo mais tarde.
- Ele permite ao usuário:
  - Reverter para estado anterior determinados arquivos ou um projeto inteiro;
  - Comparar as mudanças ao longo do tempo;
  - Ver quem modificou algo pela última vez;
  - Ver quando um problema foi inserido e quem introduziu o problema.

Controle de Versão

Introdução

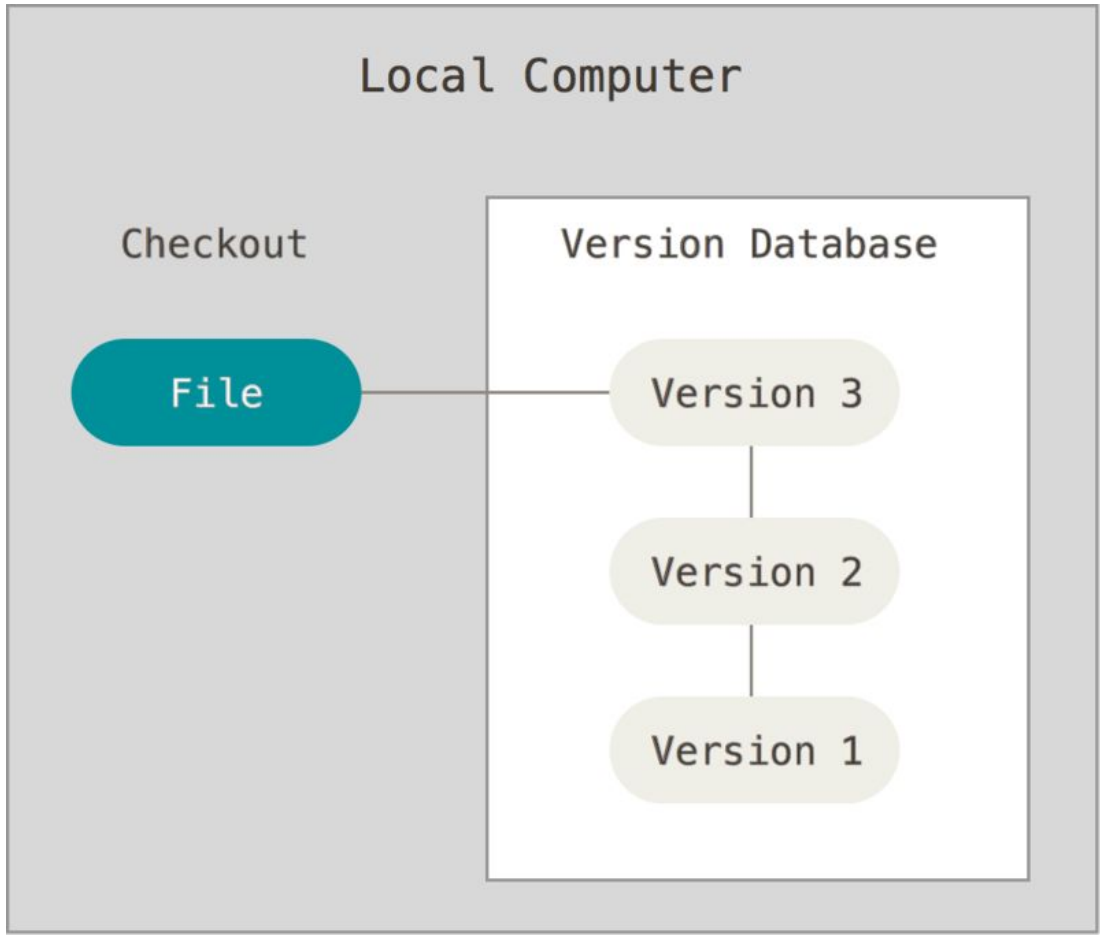
Como controlar a versão de um arquivo?

- O método de controle de versão de muitas pessoas é copiar os arquivos para outro diretório.
- Utilizar um diretório com carimbo de tempo.
- Esta abordagem é muito comum porque é muito simples.
- Altamente propensa a erros.
- É fácil esquecer em qual diretório você está e acidentalmente sobreescrever o arquivo errado ou copiar arquivos indesejados.

Controle de Versão

VCS

- **Sistemas Locais de Controle de Versão**
  - Para lidar com estes problemas, programadores desenvolveram sistemas de controle de versão locais, **VCSs** locais que tem um <u>banco de dados</u> simples que mantêm todas as alterações nos arquivos sob controle de revisão.



Controle de Versão

VCS

Sistemas Locais de Controle de Versão

- **Vantagens**
  - Implementação mais simples
- **Desvantagens**
  - Aplicável somente ao desenvolvimento local

Controle de Versão

CVCS

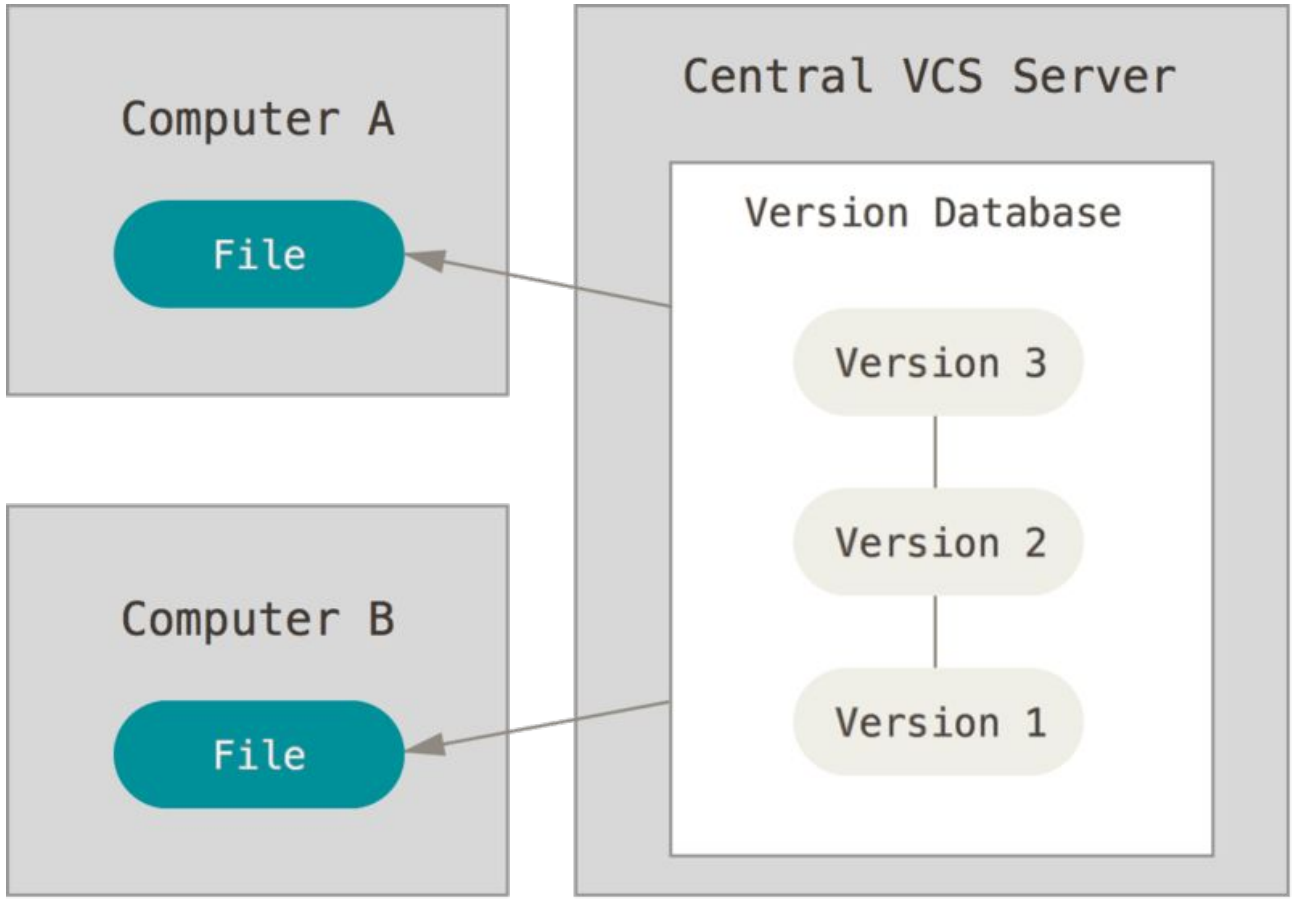
Sistemas Centralizados de Controle de Versão

- A próxima questão importante que as pessoas encontram é que elas precisam colaborar com desenvolvedores em outros sistemas.
- Para lidar com este problema, Sistemas Centralizados de Controle de Versão (CVCSs) foram desenvolvidos.
- Estes sistemas, tais como CVS, *Subversion* e *Perforce*, têm um único servidor que contém todos os arquivos de controle de versão, e um número de clientes que usam arquivos a partir desse lugar central. Por muitos anos, este tem sido o padrão para controle de versão.

Controle de Versão

CVCS

- Sistemas Centralizados de Controle de Versão



Controle de Versão

CVCS

Sistemas Centralizados de Controle de Versão

- **Vantagens**
  - Fácil controle do que cada um no projeto está fazendo.
  - Administradores tem maior controle para delegar atividades aos colaboradores.
  - É mais fácil administrar um **CVCS** do que lidar com bancos de dados locais em cada cliente.

- **Desvantagens**
  - O mais óbvio é o ponto único de falha que o servidor centralizado representa.

Controle de Versão

DVCS

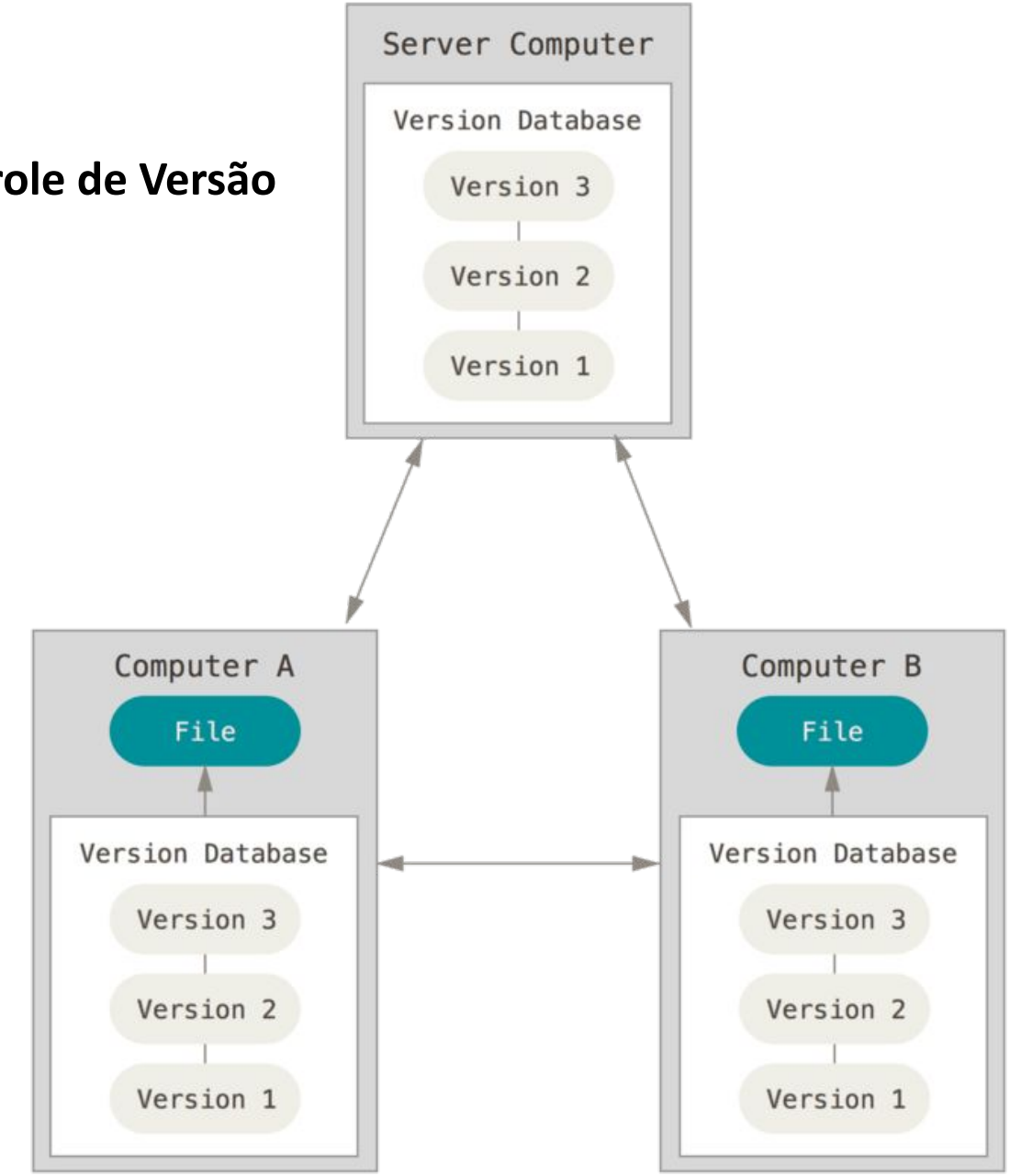
Sistemas Distribuídos de Controle de Versão

- Os Sistemas Distribuídos de Controle de Versão **DVCS** como Git, Mercurial, Bazaar ou Darcs, os clientes não somente usam o estado mais recente dos arquivos: eles **duplicam localmente** o repositório completo.
- Assim, se qualquer servidor morrer, e esses sistemas estiverem colaborando por meio dele, qualquer um dos repositórios de clientes podem ser copiado de volta para o servidor para restaurá-lo. Cada clone é de fato um backup completo de todos os dados.

Controle de Versão

INSTITUTO FEDERAL
Catarinense
Campus Camboriú

Sistemas Distribuídos de Controle de Versão



Controle de Versão

Introdução

Sistemas Distribuídos de Controle de Versão

- Além disso, muitos desses sistemas trabalham muito bem com vários **repositórios remotos**, tal que você possa colaborar em diferentes grupos de pessoas de maneiras diferentes ao mesmo tempo dentro do mesmo projeto.
- Isso permite que você configure vários tipos de **fluxos de trabalho** que não são possíveis em sistemas centralizados, como modelos hierárquicos.

INSTITUTO FEDERAL
Catarinense
Campus Camboriú

Controle de Versão

Git

Controle de Versão

Histórico

Uma Breve História do Git

O núcleo do Linux é um projeto de código aberto com um escopo bastante grande. Na maior parte da manutenção do núcleo, as mudanças no código eram compartilhadas como correções e arquivos. Em 2002, o projeto do Linux começou usar um **DVCS** proprietário chamado *BitKeeper*.

Em 2005, a relação entre a comunidade que desenvolveu o núcleo do Linux e a empresa que desenvolveu BitKeeper quebrou em pedaços, e a ferramenta passou a ser paga. Isto alertou a comunidade que desenvolvia o Linux (e especialmente Linux Torvalds, o criador do Linux) a desenvolver a sua própria ferramenta.

Controle de Versão

Histórico

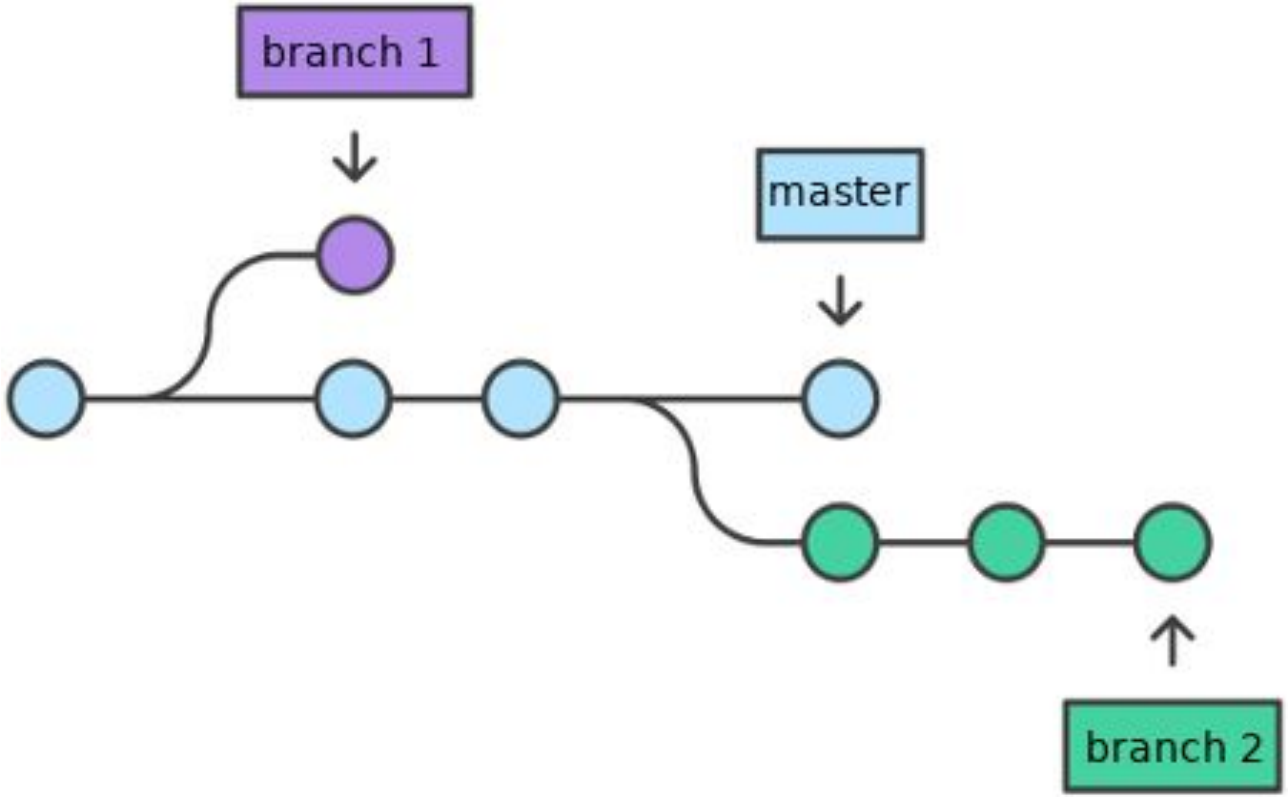
Com base nas experiências no uso do *BitKeeper,* algumas metas para o novo sistema foram definidas:

- Velocidade
- Projeto simples
- Forte suporte para desenvolvimento não-linear (milhares de ramos paralelos)
- Completamente distribuído
- Capaz de lidar com projetos grandes como o núcleo o Linux com eficiência (velocidade e tamanho dos dados)

Controle de Versão

Histórico

Desde seu nascimento em 2005, o **Git** evoluiu e amadureceu para ser fácil de usar e ainda reter essas qualidades iniciais. Ele é incrivelmente rápido, é muito eficiente com projetos grandes, e ele tem um incrível sistema de ramos para desenvolvimento não linear



Controle de Versão

Git

O Básico do Git

- Então, o que é o Git em poucas palavras?

  - Git trata seus dados como um conjunto de imagens de um sistema de arquivos em miniatura. Toda vez que você fizer um **commit**, ou salvar o estado de seu projeto no Git, ele basicamente tira uma foto desses arquivos e armazena uma referência para esse conjunto de arquivos.
  - Para ser eficiente, se os arquivos não foram alterados, Git não armazena o arquivo novamente, apenas um link para o arquivo idêntico anterior já armazenado. Git trata seus dados mais como um **fluxo do estado dos arquivos**.

Controle de Versão

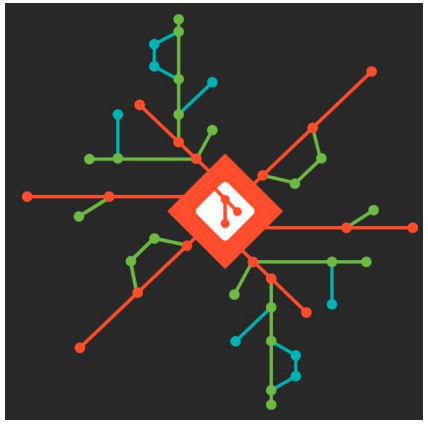
Git

- **Quase Todas as Operações são Locais**
  - A maioria das operações no Git só precisa de arquivos e recursos locais para operar, geralmente nenhuma informação é necessária de outro computador da rede.
  - Como você tem toda o histórico do projeto ali mesmo em seu disco local, a maioria das operações parecem quase instantâneas.
  - Se você quiser ver as alterações introduzidas entre a versão atual de um arquivo e o arquivo de um mês atrás, o Git faz um cálculo de diferença local.
  - Você pode fazer **commits** mesmo sem conexão de rede para enviar os arquivos. Em muitos outros sistemas, isso é impossível ou doloroso.

Controle de Versão

Git

- **Git Tem Integridade**
  - Tudo no Git passa por uma soma de verificações (**checksum**) antes de ser armazenado e é referenciado por esse **checksum** chamado um **hash** SHA-1.
  - Isto significa que é impossível mudar o conteúdo de qualquer arquivo ou pasta sem que Git saiba.
  - Esta funcionalidade está incorporada no Git nos níveis mais baixos e é parte integrante de sua filosofia. Você não perderá informação durante a <u>transferência</u> e não receberá um arquivo <u>corrompido</u> sem que o Git seja capaz de detectar.



Controle de Versão

Git

- **Os Três Estados**

  - Agora, preste atenção! Esta é a principal coisa a lembrar sobre Git. O Git tem três estados principais que seus arquivos podem estar: **committed**, **modified** e **staged**.
    - **Committed** - significa que os dados estão armazenados de forma segura em seu banco de dados local.
    - **Modified** - significa que você alterou o arquivo, mas ainda não fez o **commit** no seu banco de dados.
    - **Staged** - significa que você marcou a versão atual de um arquivo modificado para fazer parte de seu próximo **commit**.

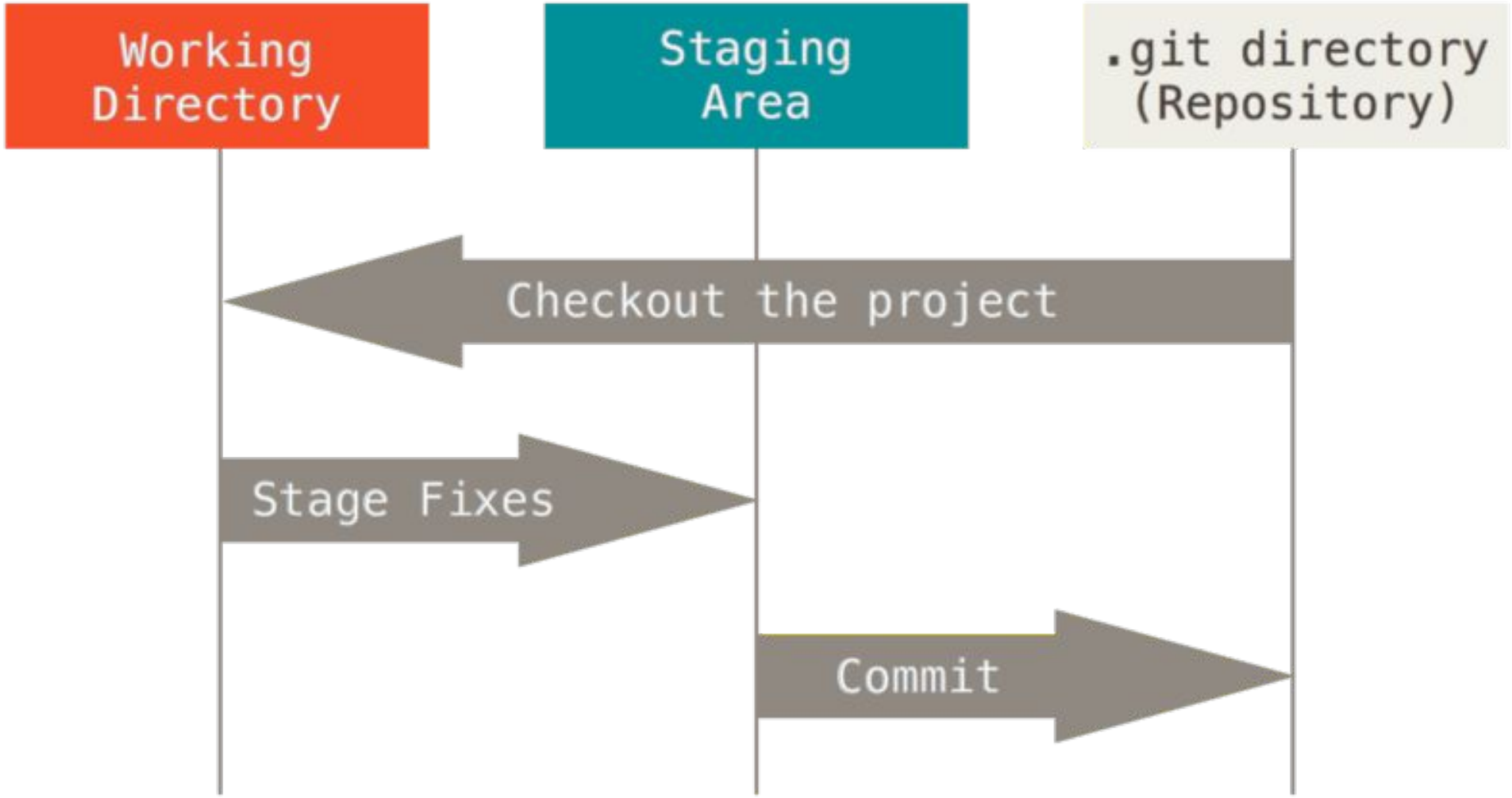
Controle de Versão

Git

- Isso nos leva a três seções principais de um projeto Git:
  - O **.git directory:** O diretório Git é onde o Git armazena os metadados e o banco de dados de objetos de seu projeto. Esta é a parte mais importante do Git, e é o que é copiado quando você clona um repositório de outro computador.
  - O **Working Directory:** O diretório de trabalho é uma simples cópia de uma versão do projeto. Esses arquivos são pegos do banco de dados compactado no diretório Git e colocados no disco para você usar ou modificar.
  - A **Staging Area:** A área de preparo é um arquivo, geralmente contido em seu diretório Git, que armazena informações sobre o que vai entrar em seu próximo **commit**. É por vezes referido como o “índice”

Controle de Versão

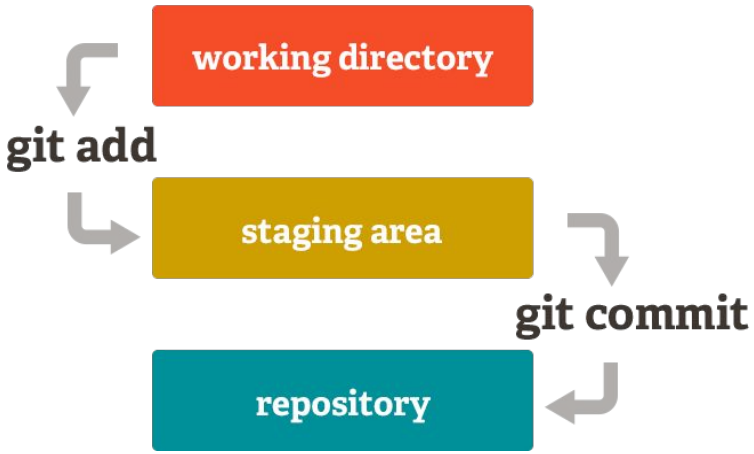
Git



INSTITUTO FEDERAL
Catarinense
Campus Camboriú

Controle de Versão

Git



- O fluxo de trabalho básico Git é algo assim:

1. Você modifica arquivos no seu diretório de trabalho.
2. Você prepara os arquivos, adicionando imagens deles à sua área de preparo.
3. Você faz **commit**, o que leva os arquivos como eles estão na área de preparo e armazena essa imagens de forma permanente para o diretório do Git.

Se uma versão específica de um arquivo está no diretório Git, é considerado **commited**. Se for modificado, mas foi adicionado à área de preparo, é considerado preparado. E se ele for alterado depois de ter sido carregado, mas não foi preparado, ele é considerado modificado.

Controle de Versão

Instalação

- **A Linha de Comando**

  - Existem várias formas diferentes de usar o Git. Existem as ferramentas originais de linha de comando, e existem várias interfaces gráficas de usuário (GUI - *Graphical User Interface*).
  - A linha de comando é o único lugar onde iremos rodar **todos** os comandos do Git - a maioria das GUI implementa somente um subconjunto das funcionalidades do Git.
  - Se você sabe como usar o Git na linha de comando, você provavelmente descobrirá como rodar versões GUI, enquanto o oposto não é necessariamente verdade.
  - Além disso, enquanto a sua escolha da interface gráfica é uma questão de gosto pessoal, **todos** os usuário terão as ferramentas de linha de comando instaladas e disponíveis.

Controle de Versão

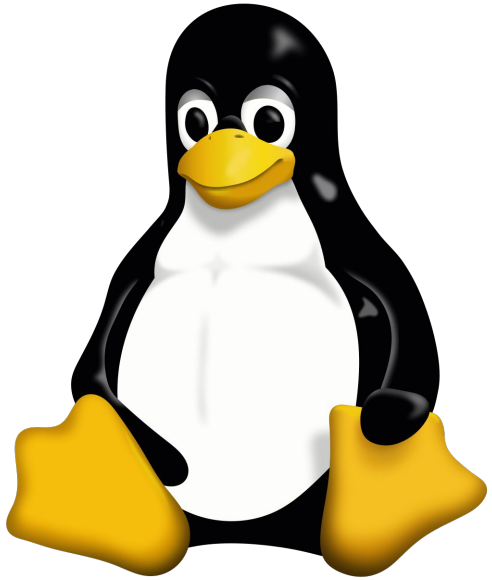
Instalação

- **Instalando no Linux**
  - Se você deseja instalar o Git no Linux através de um instalador binário, você pode geralmente fazê-lo através da ferramenta básica de gerenciamento de pacotes que vem com sua distribuição. Se você usar Fedora por exemplo, você pode usar o yum:

```
$ sudo yum install git-all
```

  - Se a distribuição for baseada em Debian como o Ubuntu, use o apt-get:

```
$ sudo apt-get install git-all
```



Controle de Versão

Instalação

- **Instalando no Windows**
  - Há também algumas maneiras de instalar o Git no Windows. A compilação mais oficial está disponível para download no site do Git. Basta ir ao http://git-scm.com/download/win e o download começará automaticamente. Note que este é um projeto chamado Git para Windows (também chamado msysGit), que é algo separado do Git; para mais informações sobre isso, vá para http://msysgit.github.io/.

Controle de Versão

Configuração

- **Configuração Inicial do Git**
  - Agora que você tem o Git em seu sistema, você deve fazer algumas coisas para personalizar o ambiente Git. Você fará isso apenas uma vez por computador e o efeito se manterá após atualizações. Você também pode mudá-las em qualquer momento rodando esses comandos novamente.
  - O Git vem com uma ferramenta chamada **`git config`** que permite ver e atribuir variáveis de configuração que controlam todos os aspectos de como o Git aparece e opera.

Controle de Versão

Configuração

- **Sua Identidade**

  - A primeira coisa que você deve fazer ao instalar Git é configurar seu nome de usuário e endereço de e-mail. Isto é importante porque cada **commit** usa esta informação, e ela é carimbada de forma imutável nos **commits** que você começa a criar:

```
$ git config --global user.name "Fulano de Tal"
$ git config --global user.email fulanodetal@exemplo.br
```

  - Reiterando, isso somente precisará ser feito uma vez se tiver usado a opção `--global`, porque o Git usará esta informação para qualquer tarefa naquele sistema. Se você quiser substituir essa informação com nome diferente para um projeto específico, você pode rodar o comando sem a opção `--global` <u>dentro daquele projeto</u>.

Controle de Versão

Configuração

- **Testando Suas Configurações**
  - Se você quiser testar as suas configurações, você pode usar o comando git config --list para listar todas as configurações que o Git conseguir encontrar naquele momento:

```
$ git config -list
user.name=Fulano de Tal
user.email=fulanodetal@exemplo.br
```

Controle de Versão

Configuração

- **Pedindo Ajuda**
  - Se você precisar de ajuda para usar o Git, há três formas de acessar a página do manual de ajuda (**manpage**) para qualquer um dos comandos Git:

```
$ git help <verb>
$ git <verb> --help
$ man git-<verb>
```

  - Por exemplo, você pode ver a **manpage** do commando config rodando

```
$ git help config
```

Estes comandos podem ser acessados de qualquer lugar, mesmo desconectado.

Controle de Versão

Fundamentos

- **Inicializando um Repositório em um Diretório Existente**
  - Para você começar a monitorar um projeto existente com Git, você deve ir para o diretório desse projeto executando:

```
$ cd /home/user/your_repository
```

  - depois digite:

```
$ git init
```

- Isso cria um novo subdiretório chamado **.git** que contém todos os arquivos necessários de seu repositório – um esqueleto de repositório Git. Neste ponto, nada em seu projeto é monitorado ainda.

Controle de Versão

Fundamentos

- **Inicializando um Repositório em um Diretório Existente**

  - Para começar a controlar o versionamento dos arquivos existentes e necessário fazer um **commit** inicial.
  - Podemos fazer isso com alguns comandos **`git add`** que especificam os arquivos que você quer monitorar, seguido de um **`git commit`**:

```
$ git add *.c
$ git add index.html
$ git commit -m 'initial project version'
```

Veremos mais a frente o que esses comandos fazem. Mas neste ponto já temos um repositório Git com arquivos monitorados e um **commit** inicial.

Controle de Versão

Fundamentos

- **Clonando um Repositório Existente**

  - Caso você queira obter a cópia de um repositório Git existente o comando para isso é **`git clone`**. Cada versão de cada arquivo no histórico do projeto é obtida por padrão quando você executa **`git clone`**.
  - Você clona um repositório com **`git clone [url]`**. Por exemplo, caso você queria clonar a biblioteca chamada *libgit2*, você pode fazer da seguinte forma:

```
$ git clone https://github.com/libgit2/libgit2
```

  - Isso cria um diretório chamado *libgit2*, inicializa um diretório **.git** dentro dele, recebe todos os dados deste repositório e deixa disponível para trabalho a cópia da última versão. Se você entrar no novo diretório *libgit2*, você verá os arquivos do projeto nele, pronto para serem editados ou utilizados.

Controle de Versão

Fundamentos

- **Clonando um Repositório Existente**
  - Caso você queira clonar o repositório em um diretório diferente de *libgit2*, é possível especificar esse diretório utilizando a opção abaixo:

```
$ git clone https://github.com/libgit2/libgit2 mylibgit
```

  - Este comando faz exatamente a mesma coisa que o anterior, mas o diretório de destino será chamado **mylibgit**.

O Git possui diversos protocolos de transferência que podemos utilizar. O exemplo anterior usa o protocolo **https://**, mas também podemos usar **git://** ou **user@server:path/to/repo.git**, que usam o protocolo de transferência **SSH**.

Controle de Versão

Fundamentos

- **Gravando Alterações em Seu Repositório**

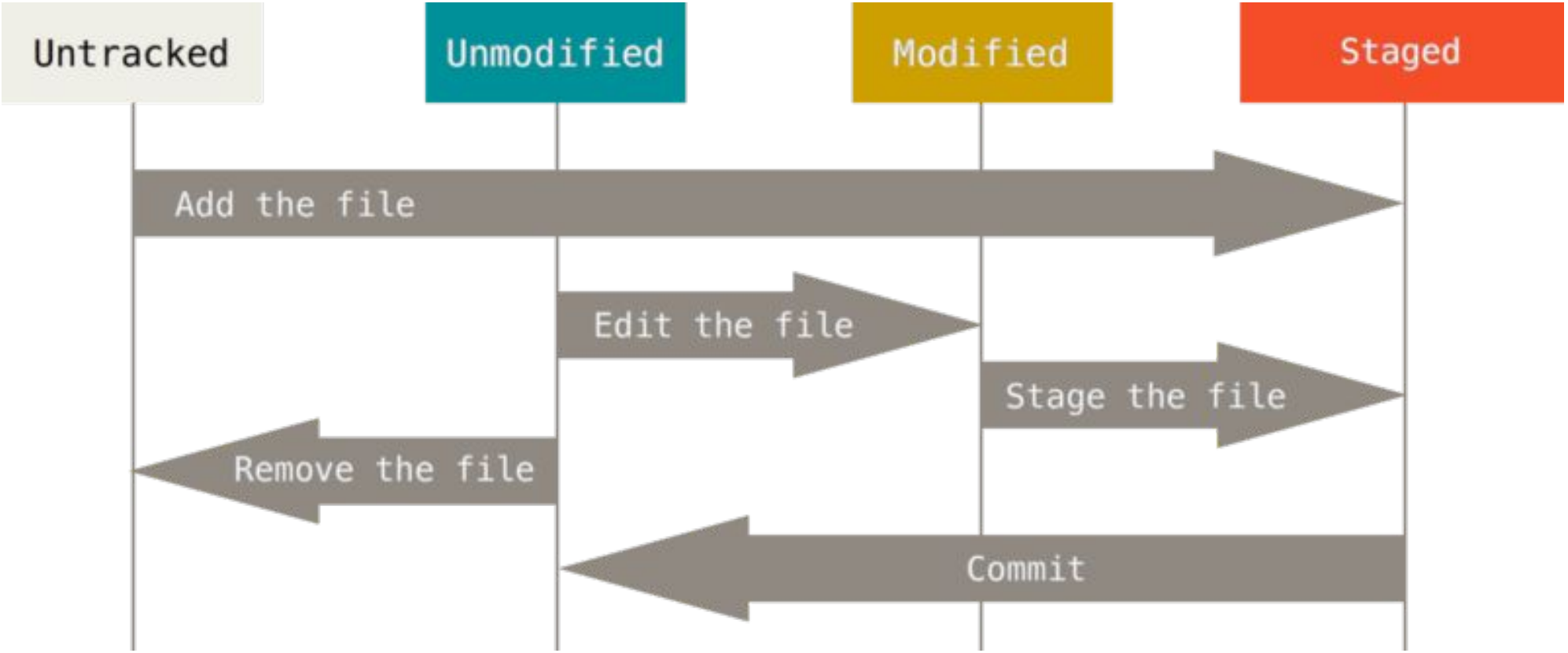
  - Uma vez criado o repositório Git para nosso projeto. Precisaremos fazer algumas alterações e adicionar **commits** dessas alterações em nosso repositório a cada vez que o projeto chegar a um estado que queiramos registrar.
  - Lembre-se que cada arquivo em seu diretório de trabalho pode estar em um dos seguintes estados:
    - Rastreado: Arquivos rastreados são arquivos que foram incluídos no último **snapshot**; eles podem ser não modificados, modificados ou preparados (adicionados ao **stage**). Arquivos rastreados são os arquivos que o Git conhece.

    - Não-rastreado: Arquivos não rastreados são todos os outros - quaisquer arquivos em seu diretório de trabalho que não foram incluídos em seu último snapshot e não estão na área de stage.

Controle de Versão

Fundamentos

- Assim que você edita alguns arquivos, Git os considera modificados, porque você os editou desde o seu último commit. Você preparar os arquivos editados e então faz commit das suas alterações, e o ciclo se repete.



Controle de Versão

Fundamentos

- **Verificando os Status de Seus Arquivos**

  - A principal ferramenta que você vai usar para determinar quais arquivos estão em qual estado é o comando git status. Se você executar esse comando imediatamente após clonar um repositório, você vai ver algo assim:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
nothing to commit, working directory clean
```

  - Isso significa que você tem um diretório de trabalho limpo - em outras palavras, nenhum de seus arquivos rastreados foi modificado.
  - Git também não está vendo nenhum arquivo não rastreado, senão eles estariam listados aqui.
  - Finalmente, o comando lhe diz qual o **branch** que você está e diz que ele não divergiu do mesmo branch no **servidor**.

Controle de Versão

Fundamentos

- **Verificando os Status de Seus Arquivos**

  - Digamos que você adiciona um novo arquivo no seu projeto, um simples arquivo *README*. Se o arquivo não existia antes, ao executar **git status**, você verá seu arquivo não rastreado da seguinte forma:

```
$ echo 'My Project' > README
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Untracked files:
  (use "git add <file>..." to include in what will be committed)

    README

nothing added to commit but untracked files present (use "git add" to track)
```

Controle de Versão

Fundamentos

- **Rastreando Arquivos Novos**
  - Para começar a rastrear um novo arquivo, você deve usar o comando git add. Para começar começar a rastrear o arquivo README, você deve executar o seguinte:

```
$ git add README
```

  - Executando o comando **status** novamente, você pode ver que seu README agora está sendo rastreado e preparado (**staged**) para o **commit**:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README
```

Controle de Versão

Fundamentos

- É possível saber que o arquivo está preparado porque ele aparece sob o título *"Changes to be committed"*. Se você fizer um **commit** neste momento, a versão do arquivo que existia no instante em que você executou `git add`, é a que será armazenada no histórico de **snapshots**.

- Você deve se lembrar que, quando executou `git init` anteriormente, em seguida, você também executou `git add (arquivos)` - isso foi para começar a rastrear os arquivos em seu diretório.

- O comando `git add` recebe o caminho de um arquivo ou de um diretório. Se for um diretório, o comando adiciona todos os arquivos contidos nesse diretório recursivamente.

- Se você modificar um arquivo depois de executar `git add`, você tem que executar `git add` de novo para por sua versão mais recente no **stage**:

- **Ignorando Arquivos**

  - Frequentemente você terá uma classe de arquivos que não quer que sejam adicionados automaticamente pelo Git e nem mesmo que ele os mostre como não-rastreados. Nesses casos, você pode criar um arquivo chamado `.gitignore`, contendo uma lista de padrões de nomes de arquivo que devem ser ignorados. Aqui está um exemplo de arquivo `.gitignore`:

```
# ignorar arquivos com extensão .a
*.a

# rastrear o arquivo lib.a, mesmo ignorando os arquivos .a acima
!lib.a

# ignorar todos os arquivos no diretório build/
build/
```

Controle de Versão

Fundamentos

- **Fazendo Commit das Suas Alterações**

  - Agora que sua área de **stage** está preparada, você pode fazer **commit** das suas alterações.
  - Lembre-se que qualquer coisa que ainda não tenha sido mandada para o **stage** — qualquer arquivo criado ou alterado e que ainda não tenha sido adicionado com `git add` — não entrará nesse **commit**.
  - O jeito mais simples de fazer **commit** é digitar `git commit`:

```
$ git commit -m "Meu primeiro Commit!"
[master 463dc4f] Story 182: Fix benchmarks for speed
 2 files changed, 2 insertions(+)
 create mode 100644 README
```

  - Note que a saída do comando fornece algumas informações: em qual **branch** foi feito o **commit** (master), seu **checksum** SHA-1 (463dc4f), quantos arquivos foram alterados e estatísticas sobre o número de linhas adicionadas e removidas.

Controle de Versão

Fundamentos

- **Removendo Arquivos**
  - Para remover um arquivo do Git, você tem que removê-lo dos seus arquivos rastreados (mais precisamente, removê-lo da sua área de **stage**) e então fazer um **commit**.
  - O comando `git rm` faz isso, e também remove o arquivo do seu diretório de trabalho para que você não o veja como um arquivo não-rastreado nas suas próximas interações.

```
git rm PROJECTS.md
rm 'PROJECTS.md'
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
(use "git reset HEAD <file>..." to unstage)

deleted: PROJECTS.md
```

  - Da próxima vez que você fizer um **commit**, o arquivo será eliminado e não será mais rastreado.

Controle de Versão

Fundamentos

- **Vendo o histórico de Commits**
  - Depois de você ter criado vários commits, ou se você clonou um repositório com um histórico de commits pré-existente, você vai provavelmente querer olhar para trás e ver o que aconteceu.
  - A ferramenta mais básica e poderosa para fazer isso é o comando `git log`.

```
$ git log
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date: Mon Mar 17 21:52:11 2008 -0700

    Meu primeiro Commit!
```

Controle de Versão

Fundamentos

- **Trabalhando de Forma Remota**

  - Para colaborar com qualquer projeto Git, você precisará saber como gerenciar seus repositórios remotos. Repositórios remotos são versões de seu repositório hospedado na Internet ou em uma rede qualquer.
  - Você pode ter vários deles, cada um dos quais geralmente é ou somente leitura ou leitura/escrita.
  - Colaborar com outras pessoas envolve o gerenciamento destes repositórios remotos, fazer:
    - **pushing** (atualizar) dados no repositório remoto.
    - **pulling** (obter) dados do repositório remoto.
  - Gerenciar repositórios remotos inclui saber como adicioná-los remotamente, remover aqueles que não são mais válidos, gerenciar vários **branches** (ramos) e definí-los como rastreados ou não e mais.

Controle de Versão

Fundamentos

- **Exibindo seus repositórios remotos**

  - Para ver quais servidores remotos você configurou, você pode executar o comando `git remote`. Ele lista os nomes abreviados de cada repositório remoto manejado que você especificou. Se você clonou seu repositório, você deve pelo menos ver **origin** (origem) – que é o nome padrão dado pelo Git ao servidor que você clonou:

```
$ git remote
origin
```

  - Você também pode especificar -v, que mostra as URLs que o Git tem armazenado pelo nome abreviado a ser usado para ler ou gravar naquele repositório remoto:

```
$ git remote -v
originhttps://github.com/schacon/ticgit (fetch)
originhttps://github.com/schacon/ticgit (push)
```

- **Adicionando Repositórios Remotos**

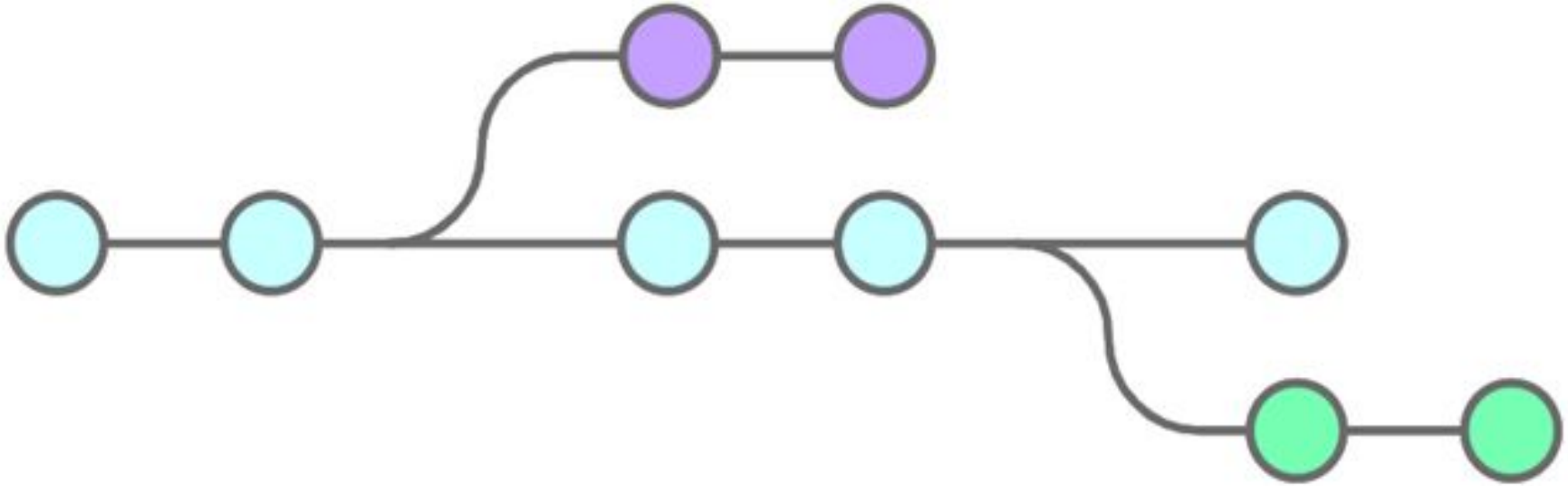
  - Nós mencionamos e demos algumas demonstrações de como o comando `clone` implicita-mente adiciona a origem (`origin`) remota para você.
  - Aqui está como adicionar um novo repositório remoto explicitamente. Para adicionar um novo repositório Git remoto como um nome curto que você pode referenciar facilmente, execute `git remote add <shortname> <url>`:

```
$ git remote add pb https://github.com/paulboone/ticgit
$ git remote -v
originhttps://github.com/schacon/ticgit (fetch)
originhttps://github.com/schacon/ticgit (push)
pb https://github.com/paulboone/ticgit (fetch)
pb https://github.com/paulboone/ticgit (push)
```

INSTITUTO FEDERAL
Catarinense
Campus Camboriú

Controle de Versão

Branching

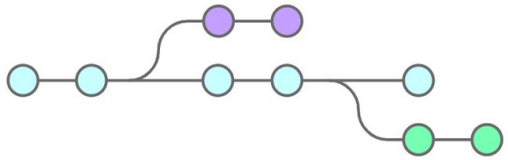


Controle de Versão

Branching

- **Ramificação em poucas palavras**

  - Quase todo VCS tem alguma forma de suporte de ramificação.
  - Ramificação significa que você diverge da linha principal de desenvolvimento e continua a trabalhar sem mexer com essa linha principal.
  - Algumas pessoas se referem ao modelo de ramificação do Git como seu "recurso matador" e certamente diferencia o Git na comunidade VCS.
  - A forma como o Git ramifica é incrivelmente leve, tornando as operações de ramificação quase instantâneas e alternando entre os branches geralmente com a mesma rapidez.



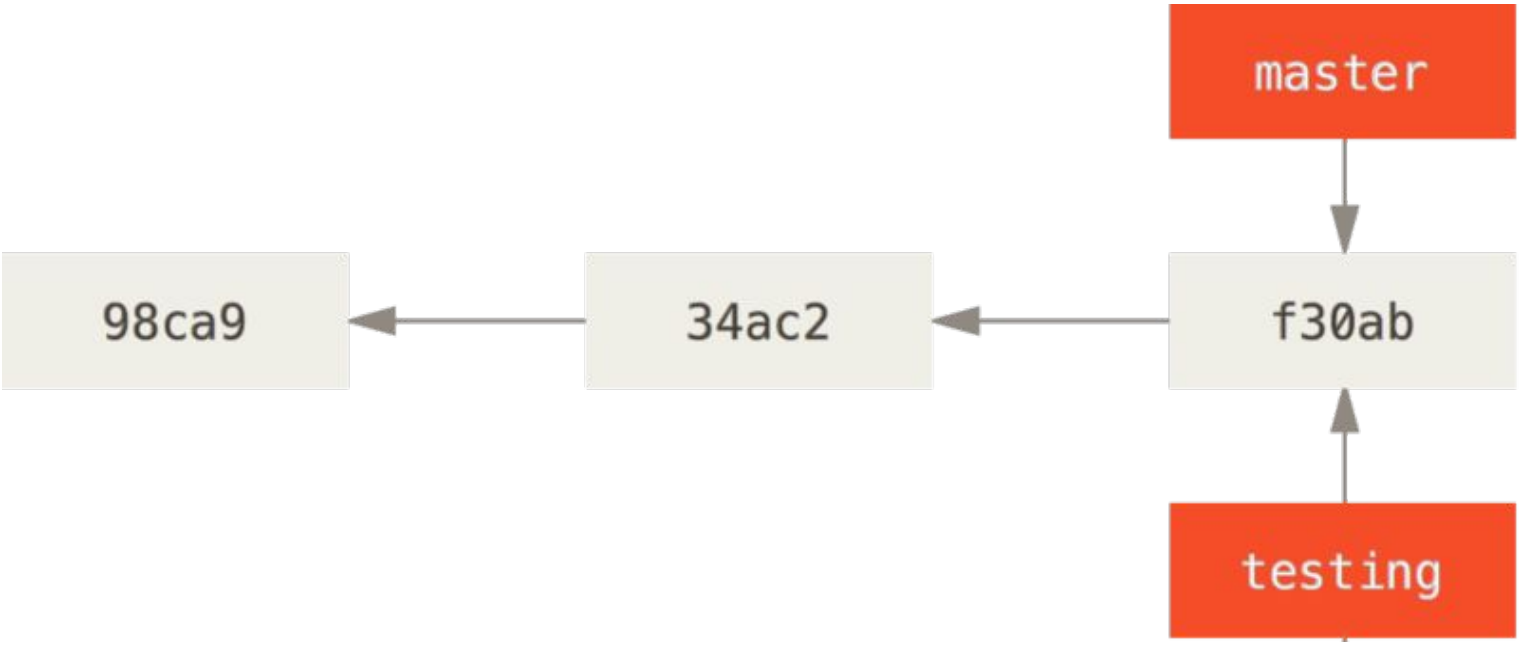
Controle de Versão

Branching

- **Criação de um novo Branch**

  - O nome do **branch** padrão no **Git** é `master`. Digamos que você crie um novo branch chamado `testing`. Você faz isso com o comando `git branch`:

```
$ git branch testing
```



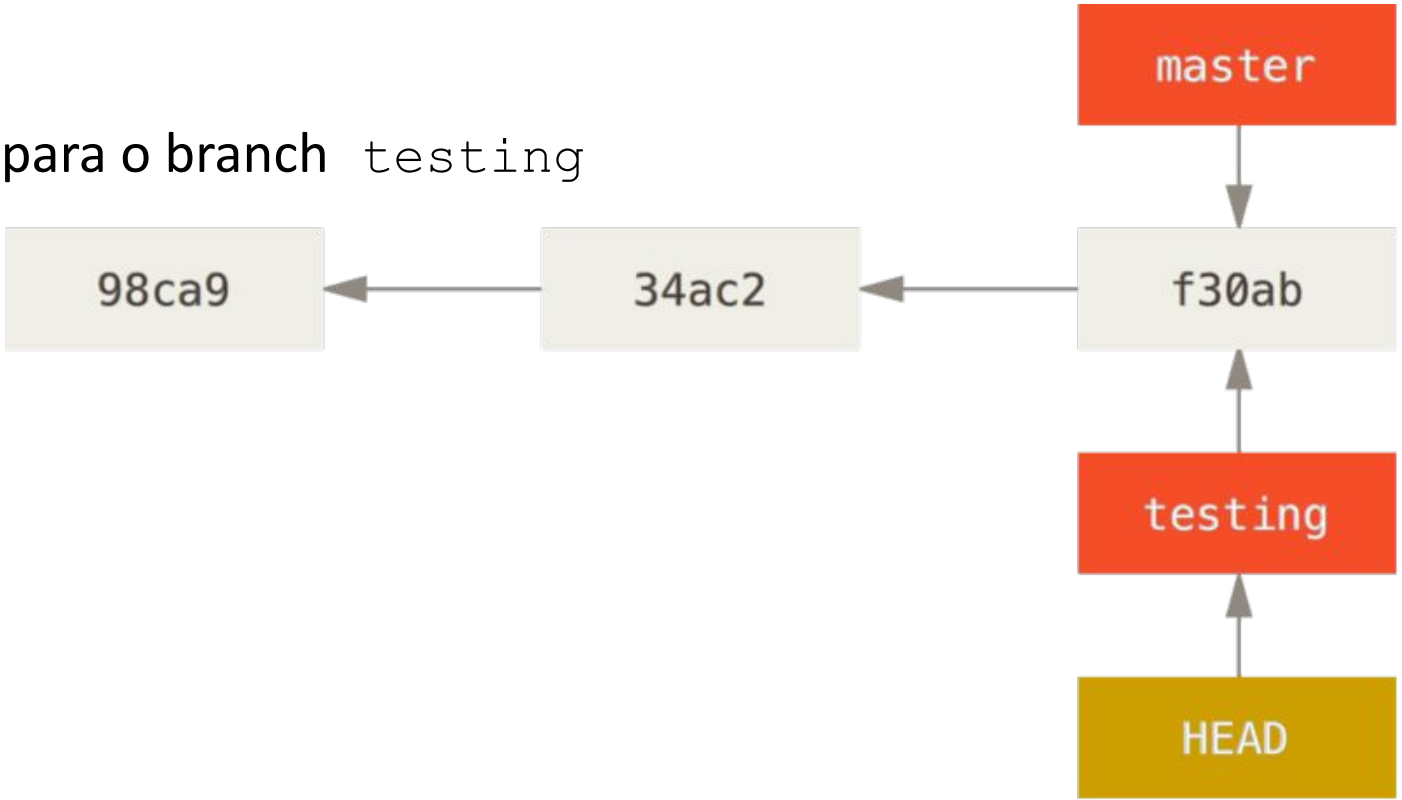
Controle de Versão

Branching

- **Alternando entre Branches**
  - Para alternar para uma ramificação existente, você executa o comando `git checkout`. Vamos mudar para o novo branch de teste:

```
$ git checkout testing
```

  - Isto faz com que a `HEAD` aponte para o branch `testing`

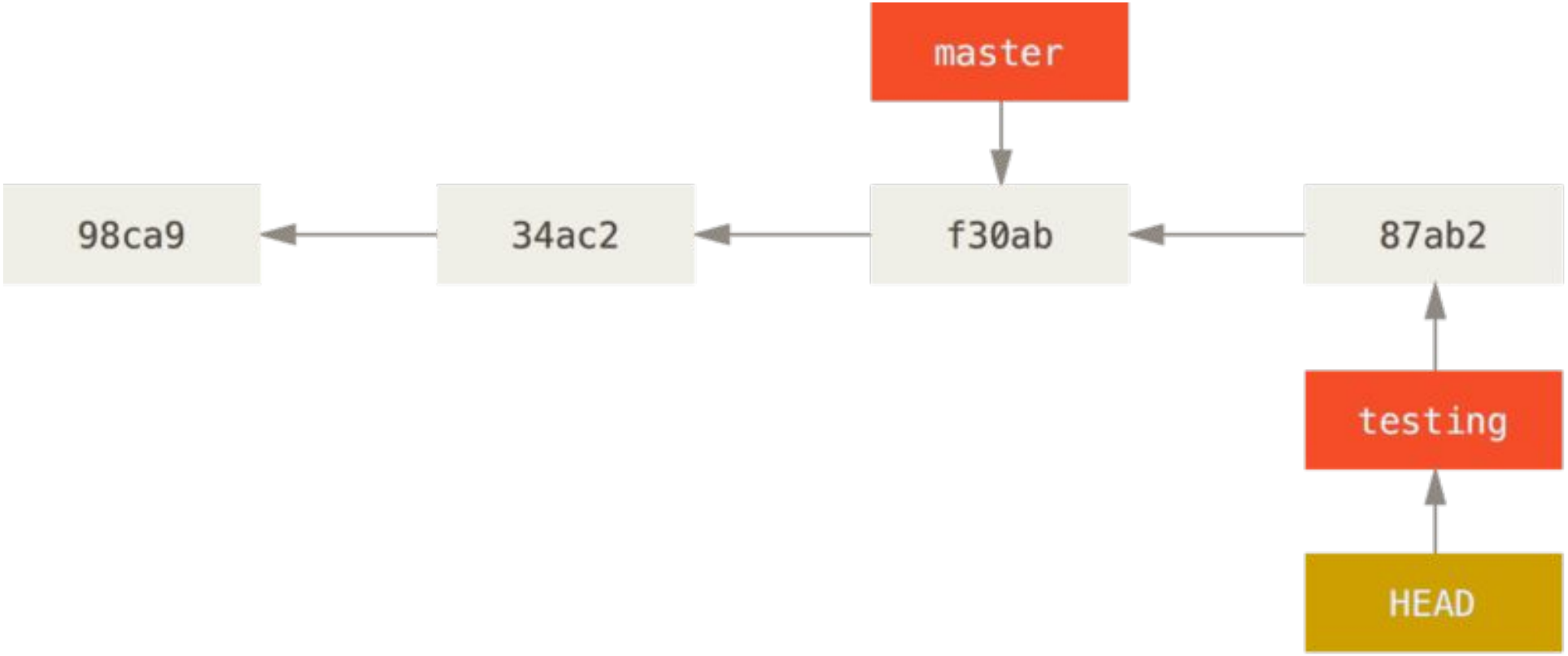


Controle de Versão

Branching

- **Alternando entre Branches**
  - Qual a importância disto? Bem, vamos fazer outro `commit`:

```
$ git commit -a -m 'made a change'
```



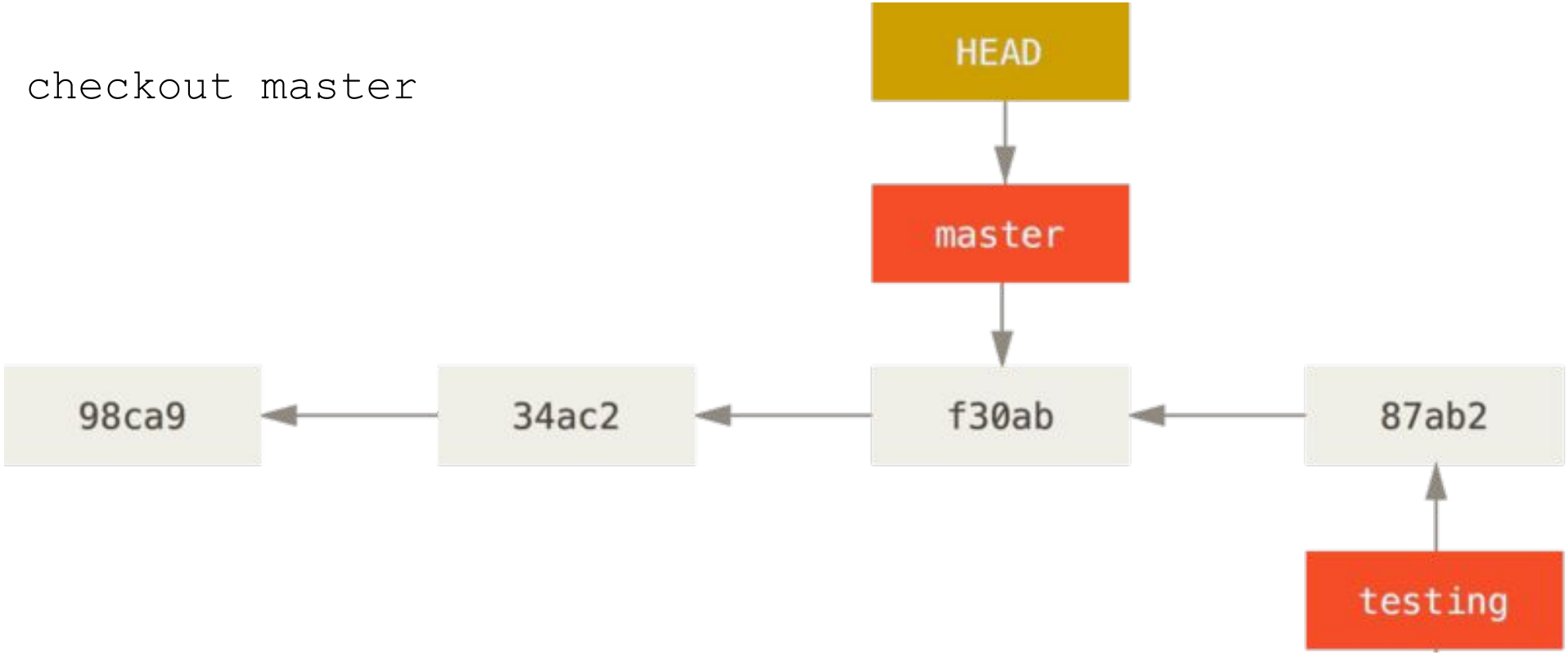
Controle de Versão

Branching

- **Alternando entre Branches**

  - Note que agora branch `testing` avançou, mas o branch `master` ainda aponta para o commit em que você estava quando executou `git checkout` para alternar branches. Vamos voltar para o branch master:

```
$ git checkout master
```

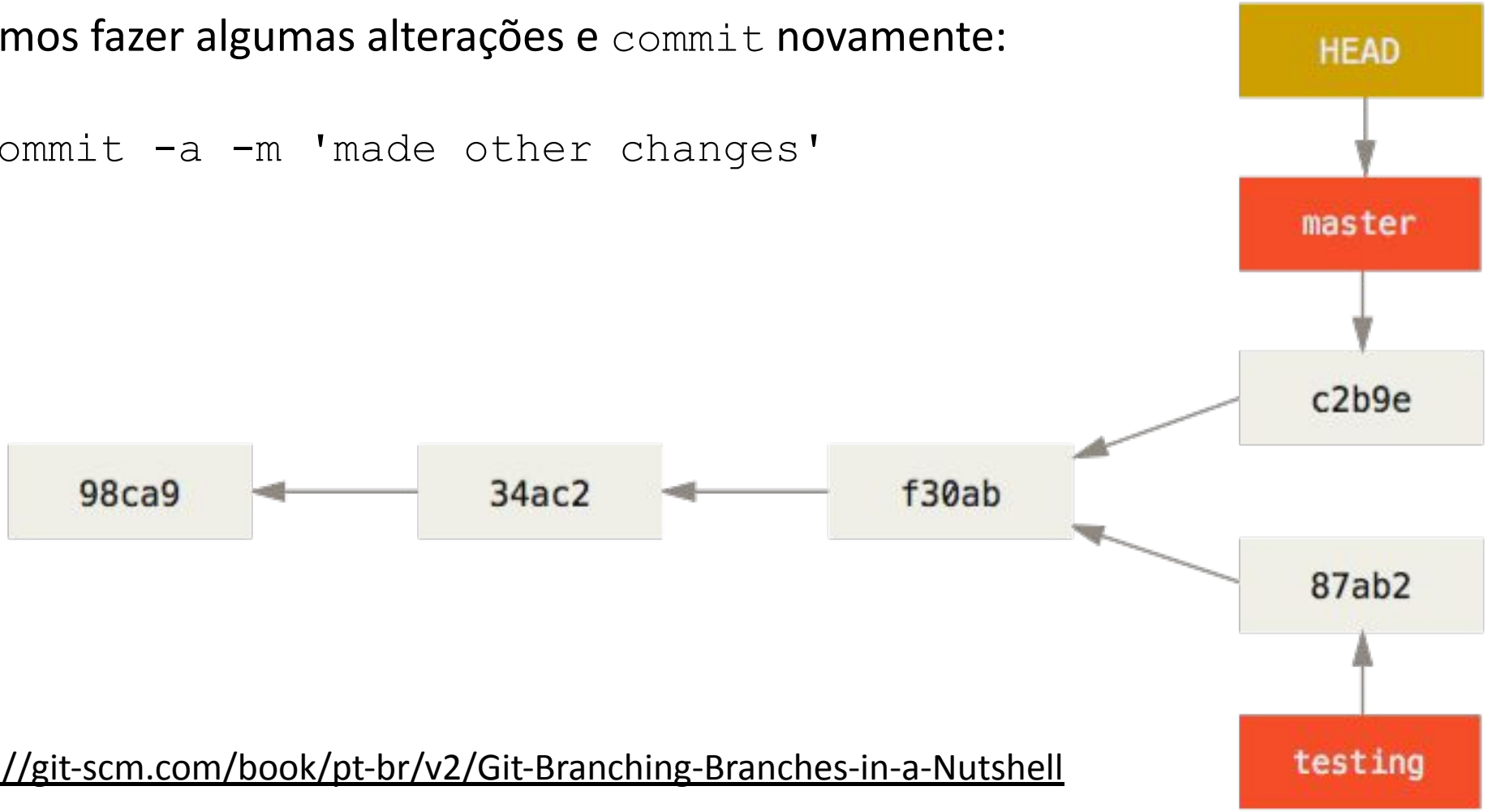


Controle de Versão

Branching

- **Alternando entre Branches**
  - Vamos fazer algumas alterações e `commit` novamente:

```
$ git commit -a -m 'made other changes'
```

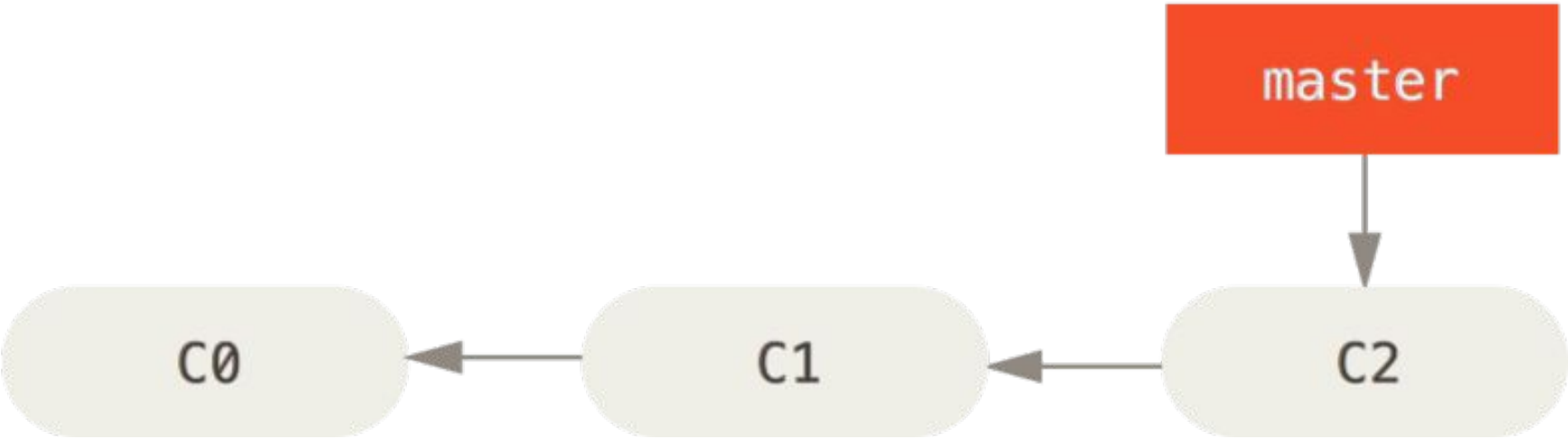


Nota: https://git-scm.com/book/pt-br/v2/Git-Branching-Branches-in-a-Nutshell

Controle de Versão

Branching

- **Fundamentos de Branches**
  - Digamos que você esteja trabalhando em seu projeto e já tenha alguns commits.



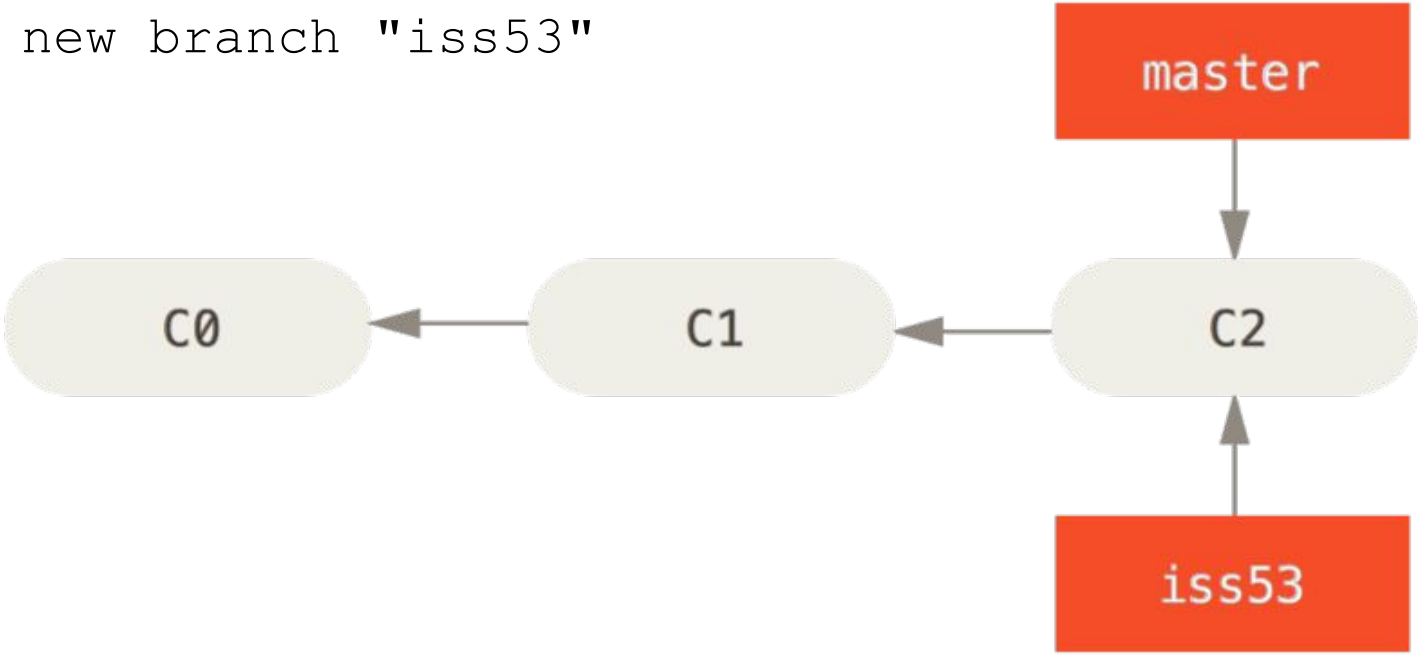
Controle de Versão

Branching

- **Fundamentos de Branches**

  - Você decidiu que trabalhará no *problema #53*. Para criar um branch e alternar para ele ao mesmo tempo, você pode executar o comando `git checkout` com a opção `-b`

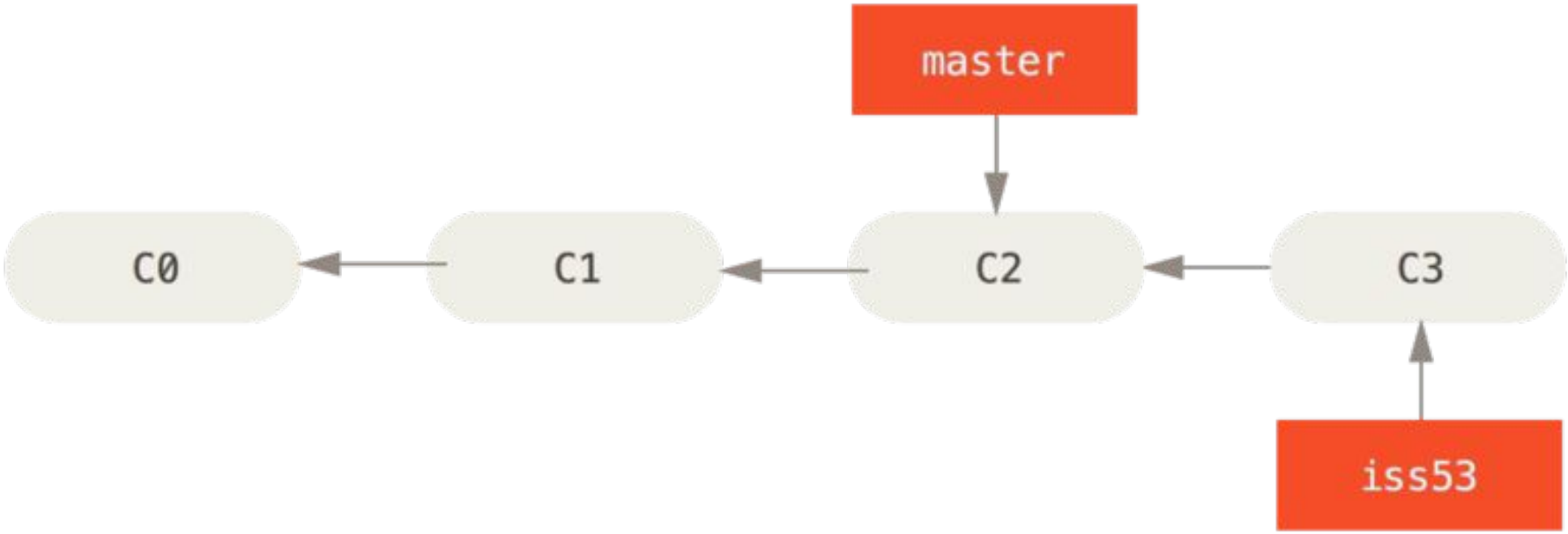
```
$ git checkout -b iss53
Switched to a new branch "iss53"
```



Controle de Versão

Branching

- **Fundamentos de Branches**
  - Você trabalha no seu site e faz alguns `commits`. Fazer isso move o `branch iss53` para frente, porque você fez `checkout` (ou seja, seu HEAD está apontando para ele):



Controle de Versão

Branching

- **Fundamentos de Branches**

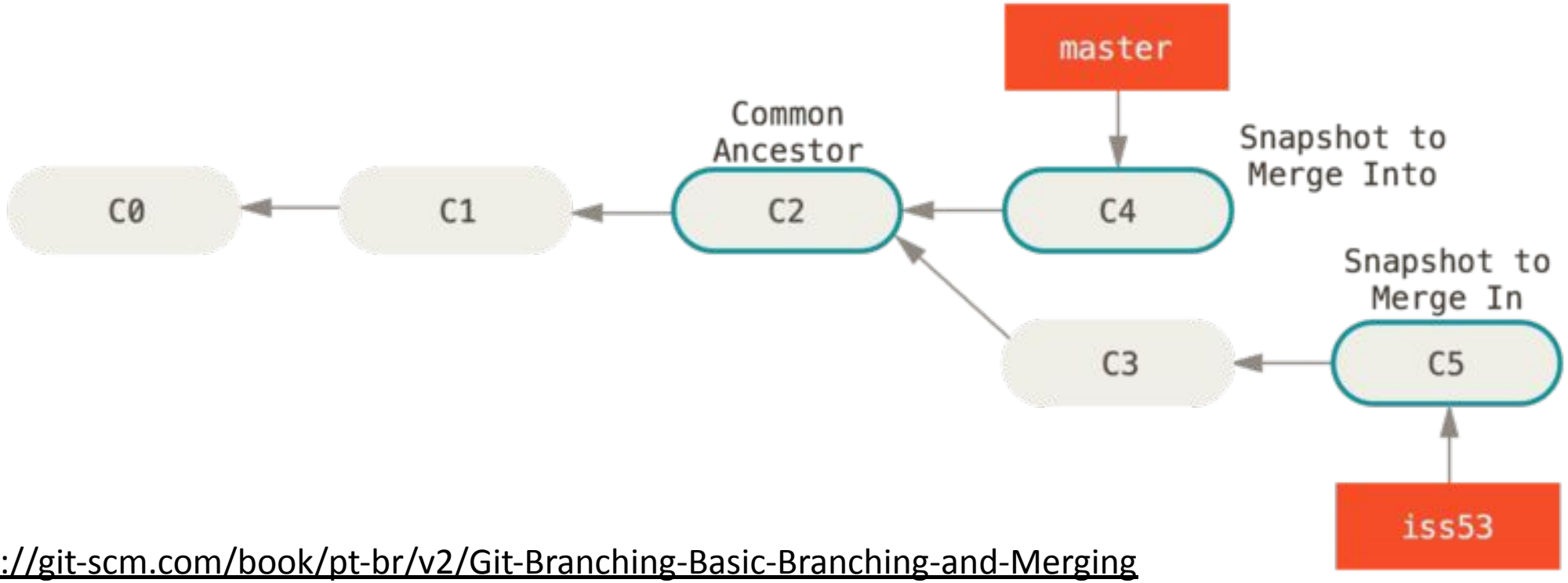
  - Suponha que você tenha decidido que seu trabalho de problema #53 está completo e pronto para ser mesclado em seu `branch master`. Para fazer isso, você mesclará seu `branch iss53` no `master`. Tudo que você precisa fazer é fazer `checkout` no branch ao qual deseja mesclar e, em seguida, executar o comando `git merge`:

```
$ git checkout master
Switched to branch 'master'
$ git merge iss53
Merge made by the 'recursive' strategy.
index.html |    1 +
1 file changed, 1 insertion(+)
```

Controle de Versão

Branching

- **Fundamentos de Branches**
  - Git faz um merge de três vias simples (thee-way merge), usando o snapshot ancestral em comum apontado pelos dois branches.



Nota: https://git-scm.com/book/pt-br/v2/Git-Branching-Basic-Branching-and-Merging

INSTITUTO FEDERAL
Catarinense
Campus Camboriú

Controle de Versão

GitHub

GitHub

Controle de Versão

GitHub

- **Configurando uma conta**

  - GitHub é o maior host para repositórios Git e é o ponto central de colaboração para milhões de desenvolvedores e projetos.
  - Uma grande porcentagem de todos os repositórios Git é hospedada no GitHub, e muitos projetos de código aberto o usam para hospedagem Git, rastreamento de problemas, revisão de código e outras coisas.
  - Então, embora não seja uma parte direta do projeto de código aberto Git, há uma boa chance de você querer ou precisar interagir com o GitHub em algum ponto ao usar o Git profissionalmente.

Controle de Versão

GitHub

- **Configurando uma conta**

  - A primeira coisa que você precisa fazer é configurar uma conta gratuita de usuário. Simplesmente visite https://github.com, escolha um nome de usuário que esteja disponível, forneça uma conta de email e uma senha, então clique no grande botão verde *"Sign up for GitHub"*

Controle de Versão

GitHub

- **Fazendo Fork de projetos**

  - Se você deseja contribuir em um projeto existente no qual você ainda não tem acesso de push, você pode dar um **“fork”** no projeto. Quando você faz “fork” de um projeto, o GitHub faz uma cópia do projeto que é inteiramente sua.

  - Dessa forma, projetos não precisam se preocupar em adicionar colaboradores para dar acesso ao **push**. Qualquer um pode dar fork no projeto, fazer push e mandar suas alterações de volta para o repositório original criando o chamado **Pull Request**, o qual falaremos a seguir.

Controle de Versão

GitHub

- **O fluxo do GitHub**

  - O GitHub é desenhado ao redor de uma fluxo particular de colaboção, centrado nas **Pull Requests**. Este fluxo funciona se você está colaborando com um time integrado em um único repositório compartilhado, com uma empresa distribuída globalmente ou com uma rede de estranhos contribuindo para o projeto com dúzias de forks:

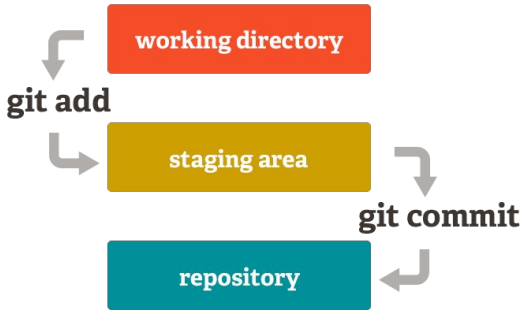
1. Faça fork do projeto.
2. Crie um branch para alterações no **master**.
3. Faça alguns commits para aprimorar o projeto.
4. Faça push dessa branch para seu projeto no GitHub
5. Abra um Pull Request no GitHub.
6. Discusta e opicionalmente continue commitando.
7. O proprietário do projeto faz merge ou fecha o Pull Request.

Nota: https://git-scm.com/book/pt-br/v2/GitHub-Contribuindo-em-um-projeto

INSTITUTO FEDERAL
Catarinense
Campus Camboriú

Revisão

- Introdução
  - Sistemas Locais de Controle de Versão
  - Sistemas Centralizados de Controle de Versão
  - Sistemas Distribuídos de Controle de Versão
- Git
  - Uma Breve História do Git
  - O básico do Git
  - Os três estados
  - As três seções principais
  - O fluxo de trabalho
  - Linha de comando
  - Instalação
  - Configuração inicial
  - Fundamentos
- *Branching*
  - Criando um novo Branch
  - Alternando entre Braches
  - Mesclando (merge) Branches
- GitHub
  - Configurando uma conta
  - Fazendo *Fork* de projetos
  - O fluxo do Github



INSTITUTO FEDERAL
Catarinense
Campus Camboriú

Referências

Bibliografia Básica

DEITEL, Paul J. Ajax, Rich. Internet applications e desenvolvimento Web para programadores. São Paulo: Pearson Prentice Hall, 2009.

SILVA, Maurício Samy. JQuery: a biblioteca do programador JavaScript. 3. ed. rev. ampl. São Paulo: Novatec, 2014.

DALL'OGLIO, Pablo. PHP: programando com orientação a objetos. 2. ed. São Paulo: Novatec, 2009.

Bibliografia Complementar

SILVA, Maurício Samy. Ajax com jQuery: requisições ajax com a simplicidade de jQuery. São Paulo: Novatec, 2009.

SOARES, Walace. Crie um Framework para sistemas web com PHP 5 e Ajax. São Paulo: Érica, 2009.

SOARES, Walace. Crie um sistema web com PHP 5 e AJAX: controle de estoque. São Paulo: Érica, 2009.

LISBOA, Flávio Gomes da Silva. Criando aplicações PHP com Zend e Dojo. 2. ed. São Paulo: Novatec, 2013.

NIEDERAUER, Juliano. Desenvolvendo websites com PHP: aprenda a criar websites dinâmicos e interativos com PHP e bancos de dados. 2. ed. rev. e atual. São Paulo: Novatec, 2011.

Bibliografia Online

https://git-scm.com/doc